

React.js

1. What is React.js?

React = JavaScript library for building user interfaces

- **Library** = focused on UI development
 - **Component** = reusable UI building block
 - **State** = data that can change
 - **Props** = data passed between components
 - **Virtual DOM** = in-memory UI representation
-

2. What is a component in React?

Component = reusable piece of UI

- **Component** = JavaScript function
 - **Props** = input to component
 - **State** = internal changing data
 - **Return** = JSX describing UI
-

3. What is JSX?

JSX = JavaScript syntax for describing UI

- **HTML-like** = looks similar to HTML
 - **JavaScript** = can use expressions
 - **Compilation** = converted into JavaScript
 - **Example** = `<h1>{name}</h1>`
-

4. What are props?

Props = data passed from parent to child

- **Props** = read-only inputs
 - **Parent** = provides data
 - **Child** = receives data
 - **Example** = `<User name="Harshit" />`
-

5. What is state?

State = data managed by a component

- **State** = changes over time
 - **setState** = updates state
 - **Update** = triggers re-render
 - **Example** = form input, counter, selected tab
-

6. Props vs state?

Props = external input, state = internal data

- **Props** = passed by parent
 - **State** = managed by component
 - **Props** = read-only
 - **State** = can be updated
 - **Both** = affect rendering
-

7. What is the Virtual DOM?

Virtual DOM = in-memory representation of UI

- **DOM** = browser's UI tree
 - **Virtual DOM** = JavaScript representation
 - **Render** = React creates updated representation
 - **Reconciliation** = React determines required DOM changes
-

8. What is reconciliation?

Reconciliation = React compares UI changes and updates DOM

- **Old tree** = previous UI representation
 - **New tree** = updated UI representation
 - **Comparison** = identifies changes
 - **Commit** = applies necessary DOM updates
-

9. What causes a React component to re-render?

Re-render = React executes component again

- **State change** = can trigger render
 - **Props change** = can trigger render
 - **Parent render** = can cause child render
 - **Context change** = can trigger render
 - **Important** = re-render doesn't always mean DOM update
-

10. What is useState?

useState = React Hook for managing component state

```
const [count, setCount] = useState(0);
```

- **count** = current state
 - **setCount** = state updater
 - **Initial value** = 0
 - **Update** = triggers re-render
-

11. What is useEffect?

useEffect = Hook for synchronizing with external systems

```
useEffect(() => {  
  fetchUsers();  
}, []);
```

- **Effect** = side effect (work outside rendering)
 - **Examples** = API calls, subscriptions, timers
 - **Dependency array** = controls when effect runs
 - **Cleanup** = removes subscriptions/timers
-

12. What is the dependency array in useEffect?

Dependency array = values that control effect execution

- `[]` = runs after initial mount
 - `[userId]` = runs when userId changes
 - **No array** = runs after every render
 - **Cleanup** = runs before effect re-runs/unmounts
-

13. What is useMemo?

useMemo = caches a computed value

```
const total = useMemo(() => calculateTotal(items), [items]);
```

- **Memoization** = caching a computed result
 - **Purpose** = avoid expensive recalculation
 - **Recalculates** = when dependencies change
 - **Don't use** = for every small calculation
-

14. What is useCallback?

useCallback = caches a function reference

```
const handleClick = useCallback(() => {
  saveUser();
}, []);
```

- **Function reference** = identity of function
 - **Purpose** = avoid unnecessary function recreation
 - **Useful** = when passing callbacks to memoized children
 - **Trade-off** = adds complexity and memory overhead
-

15. useMemo vs useCallback?

useMemo = caches value, **useCallback** = caches function

- **useMemo** = memoizes result
 - **useCallback** = memoizes function reference
 - **Use useMemo** = expensive computation
 - **Use useCallback** = stable callback reference
-

16. What is React.memo?

React.memo = prevents unnecessary child re-renders

- **Memoization** = caches rendered component result
 - **Props unchanged** = skips re-render
 - **Useful** = expensive child components
 - **Important** = only helps when props are stable
-

17. What is a controlled component?

Controlled component = form value managed by React state

```
<input value={name} onChange={e => setName(e.target.value)} />
```

- **Value** = comes from state
 - **onChange** = updates state
 - **React** = source of truth
 - **Use** = validation and dynamic forms
-

18. What is an uncontrolled component?

Uncontrolled component = form value managed by DOM

- **DOM** = source of truth
 - **ref** = accesses DOM value
 - **Less state management** = simpler for some forms
 - **Use** = simple forms or file inputs
-

19. Controlled vs uncontrolled components?

Controlled = React manages value, uncontrolled = DOM manages value

- **Controlled** = state-driven
 - **Uncontrolled** = ref-driven
 - **Controlled** = easier validation
 - **Uncontrolled** = less React state management
-

20. What is lifting state up?

Lifting state = moving shared state to common parent

- **Problem** = siblings need same data
 - **Solution** = move state to parent
 - **Parent** = owns state
 - **Children** = receive props
-

21. What is prop drilling?

Prop drilling = passing props through components unnecessarily

```
Parent
  ↓ props
Child
  ↓ props
Grandchild
```

- **Problem** = intermediate components don't need data
 - **Solution** = Context, state management, or composition
 - **Trade-off** = don't introduce global state unnecessarily
-

22. What is Context API?

Context = share data across component tree

- **Provider** = supplies value
 - **Consumer** = accesses value
 - **Avoids** = prop drilling
 - **Good for** = theme, auth, locale
 - **Not always** = replacement for all state management
-

23. What is useContext?

useContext = Hook for consuming Context value

```
const user = useContext(AuthContext);
```

- **Context** = shared data source
 - **useContext** = reads current value
 - **Re-render** = consumers update when value changes
-

24. What is useRef?

useRef = stores mutable value without causing re-render

- **DOM reference** = access DOM element
 - **Mutable value** = persists across renders
 - **State difference** = changing ref doesn't re-render
 - **Example** = timer ID, input focus
-

25. useRef vs useState?

useState = changing UI data, **useRef** = persistent non-UI data

- **useState** = update causes render
 - **useRef** = update doesn't cause render
 - **useState** = source for displayed values
 - **useRef** = DOM references/timers/previous values
-

26. What is conditional rendering?

Conditional rendering = render UI based on condition

```
{isLoggedIn ? <Dashboard /> : <Login />}
```

- **Condition** = determines UI
 - **Ternary** = inline condition
 - **&&** = render when true
 - **Use** = loading, auth, permissions
-

27. Why are keys required in React lists?

Key = uniquely identifies list item

```
users.map(user => <User key={user.id} user={user} />)
```

- **Identity** = helps React track items
- **Stable key** = should remain consistent

- **Best** = database ID
 - **Avoid** = array index when list can reorder
-

28. Why is using array index as key problematic?

Index key = can break identity when list changes

- **Insert** = items shift positions
 - **Delete** = indexes change
 - **React** = may associate state with wrong item
 - **Use index** = only for truly static lists
-

29. What is lazy loading in React?

Lazy loading = load component only when needed

```
const Dashboard = lazy(() => import("./Dashboard"));
```

- **Code splitting** = split JavaScript into chunks
 - **Initial bundle** = becomes smaller
 - **Benefit** = faster initial load
 - **Suspense** = displays loading UI
-

30. What is Suspense?

Suspense = displays fallback while something is loading

```
<Suspense fallback={<Loading />}>  
  <Dashboard />  
</Suspense>
```

- **Fallback** = temporary UI
- **Lazy component** = loaded asynchronously
- **Use** = code splitting and supported async patterns

31. What is code splitting?

Code splitting = divide application JavaScript into smaller chunks

- **Bundle** = JavaScript delivered to browser
 - **Chunk** = smaller bundle portion
 - **Lazy loading** = loads chunks when needed
 - **Benefit** = reduce initial download
-

32. How does React communicate with backend APIs?

React → HTTP request → Backend API → Response → State update

- **fetch/Axios** = HTTP client
 - **Request** = sends data
 - **Response** = receives JSON
 - **State** = stores UI-relevant data
 - **Render** = updates UI
-

33. Where should API calls be made in React?

API calls = usually triggered through effects or event handlers

- **Initial data** = `useEffect`
 - **User action** = event handler
 - **Reusable fetching** = custom Hook
 - **Complex server state** = React Query/TanStack Query
-

34. What is a custom Hook?

Custom Hook = reusable logic built using React Hooks

```
function useUsers() {  
  // fetching logic
```

```
}
```

- **Purpose** = reuse stateful logic
 - **Naming** = starts with `use`
 - **Example** = `useAuth`, `useUsers`
 - **Benefit** = keeps components focused on UI
-

35. What is server state vs client state?

Server state = data owned by backend, client state = UI-owned data

- **Server state** = users, orders, products
 - **Client state** = modal, theme, selected tab
 - **Server state** = needs fetching/caching/synchronization
 - **Client state** = usually local or global UI state
-

36. What is React Query / TanStack Query?

TanStack Query = server-state management library

- **Fetching** = handles API requests
 - **Caching** = stores server responses
 - **Refetching** = synchronizes stale data
 - **Loading/error** = manages request states
 - **Benefit** = reduces manual API state management
-

37. Redux vs Context API?

Context = sharing values, Redux = structured state management

- **Context** = simple global data
- **Redux** = complex application state
- **Redux** = predictable state updates

- **Context** = built into React
 - **Choice** = based on state complexity
-

38. What is Redux?

Redux = predictable global state management library

- **Store** = central state
 - **Action** = describes event
 - **Reducer** = calculates new state
 - **Dispatch** = sends action
 - **Selector** = reads state
-

39. What is hydration?

Hydration = attaching React behavior to server-rendered HTML

- **SSR** = HTML generated on server
 - **Browser** = receives HTML
 - **Hydration** = React attaches event handlers/state
 - **Purpose** = make server-rendered page interactive
-

40. CSR vs SSR?

CSR = browser renders, SSR = server renders initial HTML

- **CSR** = JavaScript builds UI in browser
 - **SSR** = server generates HTML
 - **CSR** = simple deployment model
 - **SSR** = can improve initial rendering and SEO
 - **Framework** = Next.js commonly provides SSR
-

41. How do you optimize a React application?

React optimization = reduce unnecessary work

- **Memoization** = cache expensive work
 - **Code splitting** = reduce initial bundle
 - **Lazy loading** = load features when needed
 - **Virtualization** = render only visible list items
 - **Stable keys** = help reconciliation
 - **Profiling** = identify actual bottlenecks
-

42. What causes unnecessary re-renders?

Unnecessary re-render = component renders without meaningful UI change

- **Parent render** = can re-render children
 - **New object** = changes reference
 - **New function** = changes reference
 - **Context update** = re-renders consumers
 - **Fix** = optimize only after profiling
-

43. How would you structure a production React application?

Production React = components + features + state + API layer

```
src/  
├─ components/  
├─ pages/  
├─ features/  
├─ hooks/  
├─ services/  
├─ store/  
├─ utils/  
└─ App.jsx
```

- **Components** = reusable UI
- **Features** = business functionality

- **Hooks** = reusable React logic
 - **Services** = API communication
 - **Store** = shared client state
-

44. How would you handle authentication in React?

Authentication = login + token/session + protected UI

- **Login** = send credentials to backend
 - **Token** = represents authenticated session
 - **Protected route** = restricts access
 - **API client** = attaches authentication credentials
 - **Logout** = clears client authentication state
-

45. What is the role of React in a full-stack architecture?

React = presentation layer of the application

```
Browser
  ↓
React
  ↓ HTTP
Backend API
  ↓
Business Logic
  ↓
Database
```

- **React** = UI and client-side state
- **Backend** = business logic and security
- **API** = communication boundary
- **Database** = persistent data
- **React should not** = own critical business rules